RAPPORT DU PROJET FLEUROPA

par
SIVASUTHAN Mathusan
BELMIR Walid

1. Présentation du Projet et Manuel Utilisateur

1.1. Présentation du contexte

Le projet Fleuropa consiste en la conception et le développement d'une application web pour la vente de fleurs. L'objectif était de lier une base de données à une interface web utilisateur.
Le site permet à Fleuropa, une chaîne de magasins de fleurs de gérer son catalogue, ses boutiques physiques, et permet aux clients de commander des bouquets en ligne pour une livraison à domicile.
On utilise :
Python > Back-end
(Flask pour “relier” python et html dans un serveur local)
(Psycopg pour “relier” python et sql pour faire les requêtes)
SQL > Base de données
HTML/CSS > Front-end

1.2. Manuel Utilisateur

Une fois le fichier ZIP décompressé vous devez:

- Ajouter le dump.sql dans votre base donnée

```
EXEMPLE DE COMMANDE :
 ~ % psql
USERNAME=# CREATE database FLEUROPA;
fleuropa=# \i 'chemin d'accés vers le dump.sql'
```

- Lancer le main.py
- Lancer : http://127.0.0.1:5000/ sur votre navigateur.

L'application est conçue pour être intuitive. Voici le parcours typique d'un utilisateur :

1. Accueil: Dès l'arrivée sur le site, l'utilisateur peut voir les "Best Sellers", les services proposés (livraison 24h, qualité garantie) et les horaires. Sur toutes les pages, en cliquant sur le nom Fleuropa, on peut revenir à l’accueil.

2. Bouquets: Pour voir l'ensemble des produits, on devrait naviguer de page en page, grâce au bouton ‘page précédente’ et ‘page suivante’. En cliquant sur le bouquet , l'utilisateur accède à sa fiche détaillée (photo, description, prix, saison), il peut aussi choisir la quantité (1-100) de bouquet a ajouté dans le panier. Il y a aussi un bouton retour au catalogue pour éviter les impasses.

3. Boutiques: affiche l'ensemble des magasins en France. En cliquant sur la ville des magasins , l'utilisateur accède à sa fiche détaillée (Code postal, Responsable, Id magasin) et le bouton voir la carte renvoie sur Google Maps, pour montrer la zone où se localise le magasin. Il y a aussi un bouton retour à la liste des boutiques pour éviter les impasses.

4. Panier: Sur la fiche produit, l'utilisateur choisit une quantité et cliquez sur "Ajouter au Panier". Il est automatiquement redirigé vers son panier où il peut voir le récapitulatif, les sous-totaux et le montant total calculé et retirer les articles non désirés. Dans Panier, vous pouvez soit continuer l'achat, valider la commande ou même revenir à l'accueil.

5. Espace Client : L'utilisateur peut s'inscrire via un formulaire ou se connecter pour retrouver ses informations et regarder les commandes passées et en cours. Un adhérent peut se connecter avec un mail et un mot de passe, pour consulter ses informations "personnel" et sur ses commandes. Un client non-adhérent doit se connecter avec le numéro de commande et le code confidentiel et ne peux que voir la commande reliée au numéro de commande. Dans les deux cas, il y a un bouton déconnecter pour quitter la session.

2. Une section technique

Nous avons examiné la demande du client et effectué un schéma entité-association, pour montrer le fonctionnement des relations entre données.

2.1. Tables Principales

La base de données fleuropa est constituée de 9 tables interconnectées. Voici le détail structurel :

- MAGASIN : Cette table centralise les points de vente physiques.
    - Attributs : IdMagasin (Clé Primaire), Lieu (Ville/Quartier), Responsable et Code Postal.
    - Spécificité : Le code postal est unique par magasin pour cette table, servant de repère géographique principal.

- CLIENT : Stocke les informations des utilisateurs.
    - Attributs : NumClient, Nom, Prenom, Mail (Unique), MDP (Mot de passe), NumTelCl et l'adresse complète.
    - Gestion des adhérents : Un booléen Adherent permet de distinguer les clients fidèles.

- BOUQUET : Le catalogue produit.
    - Attributs : IdBouquet, Modele (Nom commercial), PrixUnitaire, Description, PhotoBouquet.
    - Contraintes : Des contraintes CHECK ont été mises en place pour les champs Taille (Petit, Moyen, Grand) et Saison (Ete, Hiver...) afin de garantir l'intégrité des données dès l'insertion.

- COMMANDE : La table centrale des transactions.

    - Attributs : NumCommande, DateCommande, DateExpedition, PrixTotal.
    - Sécurité : Un CodeConfidentiel est généré pour chaque commande, permettant au client de s'identifier sans compte utilisateur complet.
    - Traçabilité : L'attribut EtatCommande suit le cycle de vie (En préparation, En cours de livraison, Livré).

- DESTINATAIRE : Dissociée de la table Client, elle permet à un utilisateur d'envoyer des fleurs à une tierce personne.
    - Attributs : NumDestinataire, Nom, Prenom, Adresse et CodePostal.

- LIVREUR : Gestion du personnel logistique.
    - Logique : Chaque livreur est associé à un CodePostal spécifique, définissant sa zone d'intervention exclusive.

- CODE_POSTAUX : Table pivot géographique.
    - Rôle : Elle relie un Code Postal à un Magasin. C'est elle qui permet de déterminer automatiquement quel magasin gère quelle zone de livraison

2.2. Relations et Associations

Deux tables d'association gèrent les relations :

1. CONTIENT (Commande - Bouquet) : Cette table détaille le contenu d'une commande. Elle possède un attribut Quantite pour savoir combien d'exemplaires d'un même bouquet ont été commandés.

2. PREND_EN_CHARGE (Livreur - Commande) : Elle permet d'assigner dynamiquement un livreur à une commande spécifique.

3. La description technique des fonctionnalités

Nous avons pu créer des vues, grâce à un bon schéma entité-association et le schéma relationnel, donc la base de données.
Parmi ces vues on a :

- top bouquet : pour montrer les “best sellers”

```
CREATE VIEW vue_top_bouquets AS
SELECT
   b.IdBouquet,
   b.Modele,
   b.Taille,
   b.PrixUnitaire,
   b.PhotoBouquet,
   COUNT(co.NumCommande) AS NombreCommandes,
```

```
   SUM(co.Quantite) AS QuantiteTotale,
   SUM(b.PrixUnitaire * co.Quantite) AS ChiffreAffaireTotal
FROM BOUQUET b
JOIN CONTIENT co ON b.IdBouquet = co.IdBouquet
GROUP BY b.IdBouquet, b.Modele, b.Taille, b.PrixUnitaire, b.PhotoBouquet
ORDER BY QuantiteTotale DESC, ChiffreAffaireTotal DESC;
```

- livreurs zones : pour récupérer les livreurs “attribué” aux magasin

```
CREATE VIEW vue_livreurs_zones AS
SELECT
   NomLiv as nomLivreur,
   NumTelLiv as numerotelephone,
   CodePostal,
   Lieu AS Magasin
FROM LIVREUR
NATURAL JOIN CODE_POSTAUX
NATURAL JOIN MAGASIN
ORDER BY Lieu, NomLiv;
```

- classement magasins: pour voir quelle magasin reçoit le plus de commande et peut être retiré des boutiques à perte

```
CREATE VIEW vue_classement_magasins AS
SELECT
   m.IdMagasin,
   m.Lieu AS NomMagasin,
   m.Responsable,
   COUNT(c.NumCommande) AS NombreVentes,
   SUM(c.PrixTotal) AS ChiffreAffaireTotal
FROM MAGASIN m
LEFT JOIN CODE_POSTAUX cp ON m.IdMagasin = cp.IdMagasin
LEFT JOIN DESTINATAIRE d ON cp.CodePostal = d.CodePostal
LEFT JOIN COMMANDE c ON d.NumDestinataire = c.NumDestinataire
GROUP BY m.IdMagasin, m.Lieu, m.Responsable
ORDER BY SUM(c.PrixTotal) DESC NULLS LAST;
```

Nous avons utilisé plusieurs requêtes sql pour avoir par exemple accès à chaque fois que l’utilisateur rentrait un formulaire des requêtes pour savoir si le numéro de commande et le code confidentiel était valide ou non , si la requête est valide, on retourne le numéro du client avec cette requête sql.

```
def recuperID(id, mdp):
   conn = connect()
   if conn is None:
       return []
   cur = conn.cursor()
```

```
    cur.execute(
        "SELECT NumCommande, CodeConfidentiel , Numclient FROM COMMANDE WHERE
NumCommande = %s AND CodeConfidentiel = %s", (id, mdp))
    resultat = cur.fetchone()
    cur.close()
    conn.close()
    return resultat
```

Nous avons décidé de stocker le panier dans session et ajouter le panier à la base de donnée une fois la commande validée, ajout_panier stocke l'ID du bouquet et la quantité dans un dictionnaire Python temporaire (session['Panier']). et la page panier sera affiché grâce à la fonction recuperer_panier_detaille :

```
def recuperer_panier_detaille(panier):
    conn = connect()
    if conn is None:
        return []
    cur = conn.cursor()
    if not panier:
        return [], 0.0
    infos = {}
    total = 0
    for bouquet, quantite in panier.items():
        cur.execute("SELECT modele, prixunitaire, taille from bouquet where
idbouquet = %s",(bouquet,))
        infos[bouquet] = cur.fetchone()
        total += quantite * infos[bouquet].prixunitaire
    return infos, total
```

La plupart des fonctions se ressemble, mais nous avons quand même rencontrer quelques problème, notamment pour les mots de passe

```
def verifiemdp(Numcommande,mdp):
    conn = connec
    if conn is None:
        return []
    cur = conn.cursor()
    password_ctx = CryptContext(schemes=['bcrypt'])
        hash_pw = password_ctx.hash(mdp)
        cur.execute(
        "SELECT NumCommande, CodeConfidentiel , Numclient FROM COMMANDE WHERE
NumCommande = %s AND CodeConfidentiel = %s", (id)  )
                resultat = cur.fetchone()
        if resultat is None:
                return None
        code_confidentiel_db, numclient = resultat
        if password_ctx.verify(mdp, code_confidentiel_db):
```

```
            return numclient
    else:
            return None
```

4. Éventuelles pistes d'améliorations

On aurait pu faire afficher les bouquets selon leurs saisons techniquement on aurait pu rajouter dans la table bouquet un attribut saison qui aurait pu avec une fonction dans db récupérer les bouquet selon leur saison et les afficher saison par saison .
On aurait pu rajouter un suivi de commande plus poussée avec des états de commande plus complets avec quel magasin livre le bouquet quel livreur le livre ect on aurait pu le faire techniquement avec plusieurs fonctions pour récupérer l'état de commande des bouquets dans la base sql .
On aurait pu rajouter un système de magasin le plus proche selon la localisation du client qui n'est pas dur a faire techniquement il fait juste récupérer l'adresse du client et chercher dans la base de données avec une requêtes sql quel magasin corresponds a quel ville a

5. Répartition des tâches

MATHUSAN 53%	WALID 47%
L'INTERFACE (HTML): ● ACCUEIL ● BOUQUETS ● BOUTIQUES ● PANIER FONCTIONNALITÉ(HTML + Flask): ● AJOUT/SUP AU PANIER ● VALIDER LA COMMANDE ● NAVIGATION DE PAGE EN PAGE DONNÉES(HTML + Psycopg + SQL): ● RÉCUP INFOS BOUQUETS ● RÉCUP INFOS MAGASINS ● GESTION DU PANIER BASE DE DONNÉES (SQL): ● CREATE TABLE/ INSERT INTO ○ MAGASIN ○ CODE POSTAUX ○ LIVREUR ○ BOUQUET ○ COMMANDE ● CREATE VIEW ○ TOP BOUQUETS	L'INTERFACE (HTML): ● CONNEXION ● INSCRIPTION FONCTIONNALITÉ(HTML + Flask): ● SESSION DU CLIENT ● GESTION DE LA SÉCURITÉ ● GESTION DES DONNÉES DU CLIENT DONNÉES(HTML + Psycopg + SQL): ● CONNEXION ● INSCRIPTION ● RÉCUPÉRATION DES INFORMATIONS CLIENT ● CHIFFRAGE DES DONNÉES DU CLIENT BASE DE DONNÉES (SQL): ● CREATE TABLE ○ CLIENT ○ DESTINATAIRE ○ CONTIENT ○ PREND_EN_CHARGE ● CREATE VIEW ○ LIVREURS ZONES ○ CLASSEMENT MAGASIN